Python Programming Lab. [Batch-B3]

Assignment Submission Details

Field	Details
Student Name *	Dilip Balu Magar
Student PRN *	125M1H055
Course Name	Python Programming Lab. (MCA31PC06)
Academic Year	2025-26 (Semester-1)
Course Teacher	Prof. Prakash Ukhalkar
Assignment Name	Assignment based on Exception Handling in Python 1
Assignment Number	Assignment 08
Submission Date *	26-11-2025

1.Write a Python program to input a number, if it is not a number generate an error message

In [3]:
```
try:
    num = int(input("Enter a number: "))
    print("Given number:", num)
except ValueError:
    print("Error: That is not a valid integer.")
```

```
Given number: 5
```

2.Write a Python program to catch ZeroDivisionError. Accept the values from the user.

In [6]:
```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print("First number is:",a)
    print("Second number is:",b)
    result = a / b
    print(f"Result:", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
except ValueError:
    print("Error: Please enter valid numbers.")
```

```
First number is: 10
Second number is: 0
Error: Cannot divide by zero.
```

3.Write a Python Program to Raise Custom Exception "PythonException" .

In [24]:
```
class PythonException(Exception):
    pass

allowed_ide = "pycharm"

try:
    ide = input("Enter your favourite Python IDE: ")
    print("Entered IDE:",ide)

    if ide.lower() != allowed_ide:
        raise PythonException("Error: Only 'PyCharm' is allowed")

    print("Great choice. PyCharm is best to code in python.")

except PythonException as e:
    print("Custom Exception:", e)
```

```
Entered IDE: vscode
Custom Exception: Error: Only 'PyCharm' is allowed
```

4.Write a Python Program to demonstrate the use of try...except...else clause..

In [7]:
```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print("First number is:",a)
    print("Second number is:",b)
    result = a / b
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
except ValueError:
    print("Error: Please enter valid numbers.")
else:
     print(f"Result:", result)
```

```
First number is: 10
Second number is: 2
Result: 5.0
```

5.Write a Python Program to demonstrate the use of try...except...finally clause

In [ ]:
```
try:
    file = open("sample.txt", "r")
    content = file.read()
    print("File contents:")
    print(content)
except FileNotFoundError:
    print("Error: File not found.")
finally: #run always not depend on exception occurs or not
    try:
        file.close()
    except NameError: #occures if file is not open
```

```
        pass   #if not handled give like NameError: name 'file' is not defined
    print("finally blocked executed")
```

```
Error: File not found.
finally blocked executed
```

6.Implement a program that handles both ValueError and TypeError exceptions when reading user input

In [13]:
```
try:
    value = input("Enter a number: ")
    print("Given data:",value)
    number = int(value)        # may raise ValueError if provided data in not integ
    result = number ** 2       # may raise TypeError if types are incompatible
    print("Square is:", result)

except ValueError:
    print("Error: given value cannot converted into integer.")

except TypeError:
    print("Error: incompatible types used in expression")
```

```
Given data: a
Error: given value cannot converted into integer.
```

7.Create a custom exception class called NegativeNumberError to signal invalid input.

In [1]:
```
class NegativeNumberError(Exception):
    #occurs if negative number is entered
    pass

try:
    num = int(input("Enter a positive number: "))
    print("Given number:", num)
    if num < 0:
        raise NegativeNumberError("Number cannot be negative.")

except NegativeNumberError as e:
    print("Error:", e)

except ValueError:
    print("Error: Please enter an integer.")
```

```
Given number: -5
Error: Number cannot be negative.
```

8. Write a function that raises a custom exception if a provided argument is not a valid file path..

In [30]:
```
import os

class InvalidFilePathError(Exception):
    #raises if file path is invalid
    pass

try:
```

```
    path = input("Enter file path: ")
    path = path.replace("\\", "/")   # to convert \ to /
    #correct path :C:\Users\dilip\OneDrive\Desktop\Desktop\FYMCA\PYTHON\Exception_H

    if not os.path.isfile(path):
        raise InvalidFilePathError("Invalid file path: " + path)

    with open(path, "rb") as f:
        print("File opened successfully.")
        print("File size:", len(f.read()), "bytes")

except InvalidFilePathError as e:
    print("Error:", e)

except FileNotFoundError:
    print("Error: File not found.")
```

```
Error: Invalid file path:  C:/Users/dilip/OneDrive/Desktop/Desktop/FYMCA/PYTHON/Hand
ling.pdf
```

9.Write a function that raises an InsufficientFundsError if a withdrawal amount exceeds the available balance.

In [ ]:
```
class InsufficientFundsError(Exception):
    #raises if current balance is less than withdrawal amount
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("Withdrawal amount exceeds current balance")
    return balance - amount

try:
    bal = float(input("Enter current balance: "))
    amt = float(input("Enter withdrawal amount: "))
    print("Current balance:",bal)
    print("Withdrawal ammount:",amt)
    new_balance = withdraw(bal, amt)
    print("Withdrawal successful. New balance:", new_balance)
except InsufficientFundsError as e:
    print("Error:", e)
except ValueError:
    print("Error: Please enter numeric values.")
```

```
Current balance: 1000000.0
Withdrawal ammount: 200000000.0
Error: Withdrawal amount exceeds current balance
```

10.Implement a program that raises a custom exception if a user enters an incorrect password multiple times. .

In [ ]:
```
class InvalidPasswordError(Exception):
    #if user enters incorrect password 3 times
    pass
```

```python
correct_password = "dilip2910"
attempts = 0

try:
    while attempts < 3:
        password = input("Enter password: ")
        print("entered password:",password)
        if password == correct_password:
            print("Login successful")
            break
        else:
            attempts += 1
            print("Incorrect password. Attempts left:", 3 - attempts)

    if attempts == 3:
        raise InvalidPasswordError("entered incorrect password 3 times.")

except InvalidPasswordError as e:
    print("Error:", e)
```

```
entered password: dilip2
Incorrect password. Attempts left: 2
entered password: dilip29
Incorrect password. Attempts left: 1
entered password: dilip291
Incorrect password. Attempts left: 0
Error: entered incorrect password 3 times.
```